

DCTwin — dcwiz Template Compliance

Proof that the code follows the dcwiz standard policy templates — 5-page summary for code handoff.
Condensed from docs/DCTwin_Template_Compliance.md (full 15-page report); verdicts and file:line evidence verified against the repository.

Compliance verdict

DCTwin adopts both dcwiz standard policy templates. RecommendTemplate is subclassed by WeeklyPlanTemplate (src/plan_weekly.py:22, import :11); TrajectoryPolicyTemplate is subclassed by AITrajectoryReplay (src/ai_trajectory_test.py:14, import :8) and BaselineTrajectory (src/baseline_policy_test.py:12, import :6). The base-class lifecycle `__init__` → `__call__` → `initialize` → `run` is honored: subclasses override `initialize()` (and `run()` for the planner) at the designated extension points (plan_weekly.py:30,67). The sample_template/ layout for configs/, data/, and models/ is mirrored: device-map JSON schemas and the dt/test/train prototxt shapes match identically or near-identically. DCTwin intentionally omits the RL-training artifacts (ai_policy_train.py, single-file hooks.py, policy/policy.pth, recommend.prototxt, train_test.prototxt) by design, because it is a search/planning optimizer, not a trained dcbrian RL policy; those responsibilities are relocated into planner/. One honest defect remains to remediate: AITrajectoryReplay calls `run(policy="ai")` without setting `self.test_collector`, so the template's `run_ai()` raises at runtime.

Compliance at a glance

Template artifact	DCTwin counterpart	Verdict
RecommendTemplate / ai_policy_test.py	src/plan_weekly.py → WeeklyPlanTemplate(RecommendTemplate)	adapted
TrajectoryPolicyTemplate / ai_trajectory_test.py	src/ai_trajectory_test.py → AITrajectoryReplay(TrajectoryPolicyTemplate)	adapted (partial: test_collector unset)
baseline_policy_test.py	src/baseline_policy_test.py → BaselineTrajectory(TrajectoryPolicyTemplate)	followed
ai_policy_train.py	none (analogue: src/fit_forecaster.py)	missing (by design)
hooks.py (single file)	distributed: planner/objective.py, kpi.py, monitor.py, oracle_worker.py, recommendation.py, broadcast.py, configs/dt/room2ite_map.json	adapted
configs/ (dt + policy)	src/configs/ (4-file dt set; test+train prototxt)	adapted (omits recommend, train_test prototxt)
data/ formats	src/data/ (EPW + schedule JSON kept; adds calibration/realized-history)	adapted
models/	src/models/ (same building.schema; idf/ subfolder; adds forecaster.pkl)	extended
policy/ (policy.pth)	none – BeamPlanner (planner/beam_search.py) is the policy	missing (by design)

Legend: followed = contract honored as-is; adapted = honored with deliberate, justified divergences; partial = contract partly honored with a gap; missing = artifact absent (here, by architectural design); extended = template honored plus domain-specific additions.

Side-by-side directory map

DCTwin mirrors the dcwiz sample_template/ layout file-for-file where its architecture overlaps, and diverges only on the RL-training slots it does not need. Marks: match = same role/shape honored; + extension = honored plus a DCTwin-specific addition; absent = not present (every absence here is by design, the planner has no trained RL policy). All src/ rows below were verified against the repo.

sample_template/ (dcwiz)	src/ (DCTwin)	Status
ai_policy_test.py	plan_weekly.py WeeklyPlanTemplate(RecommendTemplate)	— match (adapted role)
ai_trajectory_test.py	ai_trajectory_test.py — AITrajectoryReplay(TrajectoryPolicyTemplate)	match (test_collector defect)
baseline_policy_test.py	baseline_policy_test.py — BaselineTrajectory(TrajectoryPolicyTemplate)	match
ai_policy_train.py	— (closest: fit_forecaster.py)	absent (no RL training)
hooks.py (single file)	— (logic split into planner/*)	absent file / match logic
configs/dt/dt.prototxt	configs/dt/dt.prototxt	match
configs/dt/device_key_map.json	configs/dt/device_key_map.json	match
configs/dt/device_his_map.json	configs/dt/device_his_map.json	match
configs/dt/room2ite_map.json	configs/dt/room2ite_map.json	match
configs/policy/test.prototxt	configs/policy/test.prototxt	match
configs/policy/train.prototxt	configs/policy/train.prototxt	match
configs/policy/recommend.prototxt	—	absent (no RL inference)
configs/policy/train_test.prototxt	—	absent (no RL train+test)
data/weather/*.epw	data/weather/SGP_Singapore.486980_IWEC.epw	match (EPW format)
data/schedule/{acus,branches,workloads}/	data/schedule/{acus/fan_on_off,branches,workloads}/	match (acus nested)
—	data/schedule/pumps/	+ extension (CHW plant)
data/data_sample.json	data/his_data_processed.csv	match (adapted role)
—	data/{calibration.json,calibration_history.json,realized_history.csv}	+ extension (dual-loop)
models/building.json	models/building.json	match
models/building.idf	models/idf/building.idf	match (subfolder)
—	models/forecaster.pkl	+ extension (statistical forecaster)
policy/policy.pth	— (planner/beam_search.py::BeamPlanner)	absent (algorithmic policy)

Evidence by template

DCTwin subclasses both dcwiz standard policy templates and mirrors the sample_template layout for configs/, data/, and models/. Each base-class contract (___init___ → ___call___ → initialize → run) is honored at the designated extension points. All DCTwin-side (src/...) citations below were re-verified against the repo. Template-side citations are taken as given from the source report (external dcwiz/dcbrain sample). Verdict vocabulary: followed = contract honored as-is; adapted = honored with justified divergences; partial = honored with a gap; missing = absent by design; extended = honored plus domain additions.

(a) RecommendTemplate → WeeklyPlanTemplate [adapted]

WeeklyPlanTemplate subclasses RecommendTemplate and imports it from dcwiz_policy_template, overriding initialize() to wire forecaster/oracle/beam-planner and fully replacing run() to call run_weekly_plan(). The base run() (which expects a reactive dcbrain policy and would call log_recommendations()) is never reached – a justified divergence for a deterministic batch planner, not a contract violation.

Claim	DCTwin ref (file:line)	Note
Subclasses RecommendTemplate; from dcwiz_policy_template	src/plan_weekly.py:11,22	import at :11, class WeeklyPlanTemplate(RecommendTemplate) at :22
initialize() overridden with concrete impl	src/plan_weekly.py:30	Base raises NotImplementedError; DCTwin sets forecaster, ParallelEnvOracle, BeamPlanner; self.policy stub at :63
run() replaced to call run_weekly_plan()	src/plan_weekly.py:67-86	Calls run_weekly_plan() at :73, write_recommendation() at :83-84; base policy call bypassed, log_recommendations() never reached

(b) TrajectoryPolicyTemplate → AITrajectoryReplay [adapted, partial]

AITrajectoryReplay subclasses TrajectoryPolicyTemplate and overrides initialize() to set self.env and self.act (the fixed replay action expanded from the recommendation JSON), as the contract requires. The gap: `__call__` is invoked with `policy="ai"` but `self.test_collector` is never set, so the template's `run_ai()` raises `ValueError("Test collector is not set")` at runtime. Fix: set `self.test_collector` in `initialize()`, or call with `policy="baseline"` (the fixed-action replay is baseline-shaped anyway). This is the one honest defect; everything else here is followed.

Claim	DCTwin ref (file:line)	Note
Subclasses TrajectoryPolicyTemplate	src/ai_trajectory_test.py:14	import at :8; class AITrajectoryReplay(TrajectoryPolicyTemplate)
initialize() sets self.env + self.act (fixed replay)	src/ai_trajectory_test.py:17-28	self.env via <code>dctwin.make_env</code> at :20; self.act = <code>mapper_from_env(self.env).expand(setpoints)</code> at :28
GAP: <code>policy="ai"</code> with no <code>self.test_collector</code> → <code>run_ai()</code> raises	src/ai_trajectory_test.py:36	<code>__call__(policy="ai", ...)</code> dispatches to <code>run_ai()</code> , which rejects an unset <code>test_collector</code> – crashes as written

(c) TrajectoryPolicyTemplate → BaselineTrajectory [followed]

BaselineTrajectory honors the contract end-to-end: subclass, initialize() sets self.env and a conservative self.act (coolest SAT/CHW, maximum airflow), and the base `run_baseline()` runs unchanged via `policy="baseline"`. No gaps; the only difference from the template is a benign encoding choice (parametrized Setpoints + broadcast vs hand-built per-zone arrays), with identical physical semantics.

Claim	DCTwin ref (file:line)	Note
Subclasses TrajectoryPolicyTemplate	src/baseline_policy_test.py:12	import at :6; class BaselineTrajectory(TrajectoryPolicyTemplate)
initialize() sets conservative self.act (coolest SAT/CHW, max airflow)	src/baseline_policy_test.py:15-20	Setpoints(sat_c=s.sat.lb, flow_kg_s=s.flow.ub, chwst_c=s.chwst.lb) from DEFAULT_SEARCH_SPACE (types.py:81-85: sat[20,26], flow[4.8,13.8], chwst[13,19]); expanded at :20
Base <code>run_baseline()</code> via <code>policy="baseline"</code>	src/baseline_policy_test.py:27	<code>__call__(policy="baseline", ...)</code> routes to unmodified base <code>run_baseline()</code>

Shared layouts honored: configs, data, models

The `configs/`, `data/`, and `models/` trees mirror the `dcwiz` `sample_template` layout. Shared JSON schemas and protobuf shapes are honored exactly; DCTwin adds domain-specific files for its dual loop and omits only the RL-only configs. Every `src/` fact below was verified against the repository.

configs/ — JSON schemas and prototxt shapes match

Artifact	Evidence (verified)
configs/dt/device_key_map.json	Identical set of 13 top-level device keys (acus, chillers, ites, zones, cooling towers, ...); same schema, fewer device instances
zones / ites sub-schemas	zones = {air temperature, air relative humidity, ite power}; ites = {inlet dry-bulb temp, inlet RH, cpu/fan/ups power} — identical
device_his_map.json, room2ite_map.json	Same structure; room2ite = {room → {ite → {wattsPerUnit, numberOfUnits, totalWatts}}}
dt.prototxt, test.prototxt, train.prototxt	Same protobuf sections (eplus_env_config; logging_config + policy_config + test_collector / off_policy_trainer)
recommend.prototxt, train_test.prototxt	Omitted by design (RL-only inference / combined train+test)

data/ — formats kept, dual-loop files added

Artifact	Evidence (verified)
weather/*.epw	Identical ASHRAE EPW header format (SGP_Singapore.486980_IWEC.epw)
schedule/ {acus,branches,workloads}/	JSON-array schedules honored (acus nested under fan_on_off/); + pumps/ added for the CHW plant
data_sample.json → his_data_processed.csv	Static policy sample replaced by per-15-min telemetry (adapted role)
+ calibration.json, calibration_history.json, realized_history.csv	Dual-loop additions: bias/sigma, paired predicted/realized weeks, weekly KPI history

models/ — schema-compliant, plus a forecaster

Artifact	Evidence (verified)
building.json	Identical top-level keys: constructions, geometry, inputs, meta, models
building.idf → idf/building.idf	Moved into an idf/ subfolder (logical grouping); same IDF format
+ forecaster.pkl (655 B config)	Statistical forecaster config replaces policy.pth (2.9 MB neural weights) — model-prep slot

Why DCTwin diverges (by design)

One architectural fact explains every "missing" verdict: DCTwin is a search/planning optimizer, not a trained RL agent. The dcwiz template assumes a dcbrain SAC policy persisted as policy/policy.pth, fed per-step rewards and states through hooks.py, and trained by ai_policy_train.py. DCTwin instead beam-searches three global setpoints (SAT, airflow, CHWST), scoring every candidate with a full-week EnergyPlus oracle run. That swap removes the RL training pipeline and relocates its responsibilities into the planner/ package and JSON config, verified present in src/ (all targets below confirmed).

Template (RL) artifact	DCTwin replacement	Where it lives
policy/policy.pth (trained weights)	BeamPlanner (in-code algorithm)	planner/beam_search.py:70
ai_policy_train.py (trainer.run())	one-shot model/config prep	fit_forecaster.py:42 → models/forecaster.pkl
hooks.get_reward_fn() (per-step scalar)	weekly objective rollout	planner/objective.py:47 + planner/kpi.py
hooks.get_current_state()	observation discovery + sampling	planner/monitor.py + planner/oracle_worker.py
hooks.log_recommendations()	recommendation builder/serializer	planner/recommendation.py
zone maps (zone_acu_dict / model_map)	global→actuator broadcast plant model	planner/broadcast.py + planner/plant.py + configs/dt/room2ite_map.json
recommend.prototxt / train_test.prototxt	Python BeamConfig dataclass	plan_weekly.py:56-60 (def beam_search.py:33)

One item to remediate

One honest defect. AITrajectoryReplay (src/ai_trajectory_test.py:14) overrides initialize() at :17-28 to set self.env and self.act, but never sets self.test_collector, then is invoked with policy="ai" at :36. The template's run_

`ai()` requires a test collector and raises `ValueError("Test collector is not set")`, so the call crashes as written. Fix, two options: set `self.test_collector` in `initialize()`, OR call with `policy="baseline"` (the fixed-action replay is baseline-shaped anyway). `BaselineTrajectory` (`src/baseline_policy_test.py:12`) is unaffected and fully followed.

Conclusion

- `RecommendTemplate`: followed via adaptation. `WeeklyPlanTemplate` (`plan_weekly.py:22`) imports from `dcwiz_policy_template (:11)`, overrides `initialize() (:30)` and `run() (:67)` at the designated extension points, and honors the `__init__`/`__call__` lifecycle.
- `TrajectoryPolicyTemplate`: followed for baseline, partial for AI replay. `BaselineTrajectory` honors the contract end-to-end via `policy="baseline"`; `AITrajectoryReplay` honors structure + `initialize()` but has the `test_collector` gap above.
- `configs/`, `data/`, `models/`: honored with extensions. JSON device-map schemas, EPW weather, schedule-folder convention, and `dt/test/train` prototxt shapes all match; DCTwin adds calibration/realized-history, pumps/, and `forecaster.pkl` without breaking shared schemas.
- RL-training artifacts (`ai_policy_train.py`, single-file `hooks.py`, `policy/policy.pth`, `recommend.prototxt`, `train_test.prototxt`): intentionally absent, replaced by `fit_forecaster.py`, the `planner/` package, and `BeamPlanner`.
- Net: the template contracts (base-class subclassing, the `initialize/run/__call__` lifecycle, the `config/data/model` layouts) are honored. One gap to fix: the `AITrajectoryReplay` `test_collector` omission; everything else is a justified architectural adaptation.